Pimpri Chinchwad Education Trust's

PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

(An Autonomous Institute affiliated to Savitribai Phule Pune University)

DEPARTMENT OF INFORMATION TECHNOLOGY

MINI PROJECT REPORT

on

"Personal Expense Tracker - Spring Boot & React"

Submitted by

Student 1:
Name: Arpita Rahul Patki
PRN No.: 123B1F001
Class & Division: TY – IT (A)

Student 2:
Name: Varad Sushant Deshmukh
PRN No.: 123B1F022
Class & Division: TY – IT (A)

Faculty Guide: Mrs.Tanuja Patankar

Academic Year: 2025–26

Table of Contents

1. Introduction

1.1 Background

Managing personal finances is a fundamental challenge for individuals in today's fast-paced world. Traditional methods such as spreadsheets or pen-and-paper tracking are often cumbersome, error-prone, and lack real-time analytics. The Personal Expense Tracker addresses these limitations by providing a centralized, secure, and feature-rich full-stack web application built using modern technologies.

This application integrates React.js on the frontend and Spring Boot on the backend, connected through RESTful APIs and backed by a MySQL database. It provides users with a seamless experience to log, categorize, visualize, and analyze their expenses across devices.

1.2 Problem Statement

Current expense tracking approaches face several key challenges:

- Traditional expense tracking methods are time-consuming and error-prone
- Lack of a centralized platform for personal financial management
- Difficulty in analyzing spending patterns across categories and time periods
- No real-time synchronization of data across devices
- Manual categorization of expenses leading to inconsistencies
- Absence of visual representations for financial data
- Security concerns with personal financial information being stored insecurely

The proposed solution is a modern, secure web application that provides centralized expense management, automated categorization, visual analytics, and secure authentication to address all the above challenges.

1.3 Objectives

Primary Objectives:

- Develop a user-friendly web application for personal expense management
- Implement a robust backend REST API using Spring Boot 3.2 and Java 21
- Create a responsive frontend using React.js 18.2 and Tailwind CSS
- Establish secure user authentication and authorization using JWT
- Design an efficient database schema for transaction management using MySQL

Secondary Objectives:

- Provide visual analytics through interactive pie charts and reports
- Implement full CRUD operations for expense management
- Enable expense filtering, sorting, and search capabilities
- Create comprehensive API documentation
- Ensure cross-browser compatibility and responsive design
- Implement thorough error handling and input validation

2. Frontend Development

2.1 UI/UX Design

The frontend is built using React.js 18.2 combined with Tailwind CSS for a utility-first approach to styling. The design follows a mobile-first philosophy with adaptive layouts that ensure a consistent and engaging experience across desktop, tablet, and mobile devices.

Key design principles applied include:

- Clean and minimalist interface with a structured dashboard layout
- Consistent color scheme using blue tones for trust and professionalism
- Intuitive navigation with sidebar and header components
- Accessible form elements with proper labels and validation feedback
- Interactive charts via Recharts for visual expense analysis

2.2 Components and Structure

The React application is organized into clearly defined modules:

Authentication Module:

- LoginComponent: Secure user login interface with form validation
- RegisterComponent: User registration form with input verification
- ProtectedRoute: Route guard for authenticated-only access

Dashboard Module:

- DashboardComponent: Main overview showing totals and recent activity
- SummaryCards: Financial summary tiles (total expenses, category totals)
- QuickStats: Recent transaction statistics widgets

Expense Management Module:

- ExpenseFormComponent: Add/Edit expense form
- ExpenseListComponent: Display all expenses with search and filter
- ExpenseTableComponent: Sortable tabular view of expenses
- FilterComponent: Filter panel (by date, category, amount)

Analytics Module:

- ExpenseChartComponent: Interactive pie chart visualization
- ReportsComponent: Monthly and weekly expense summaries
- CategoryBreakdown: Category-wise spending analysis

2.3 Routing and Navigation

React Router DOM v7.14.0 is used for client-side routing. Routes are protected using a ProtectedRoute component that validates the JWT token before rendering. Navigation is managed through a responsive sidebar with links to Dashboard, Expenses, Analytics, and Profile pages.

2.4 Responsiveness

Tailwind CSS utility classes and responsive grid/flexbox layouts ensure the application adapts seamlessly to any screen size. Touch-friendly interactions are implemented for mobile users. The application has been tested across Chrome, Firefox, and Edge browsers.

3. Backend Development

3.1 Spring Boot Architecture

The backend follows a layered three-tier architecture built on Spring Boot 3.2 with Java 21:

- Controller Layer: Handles incoming HTTP requests and maps them to service operations
- Service Layer: Contains business logic and orchestrates repository calls
- Repository Layer: Manages data persistence using Spring Data JPA and Hibernate
- Entity Layer: Data models corresponding to MySQL database tables

The architecture diagram below illustrates the complete data flow from the React frontend through the network layer to the Spring Boot backend and finally the MySQL database:

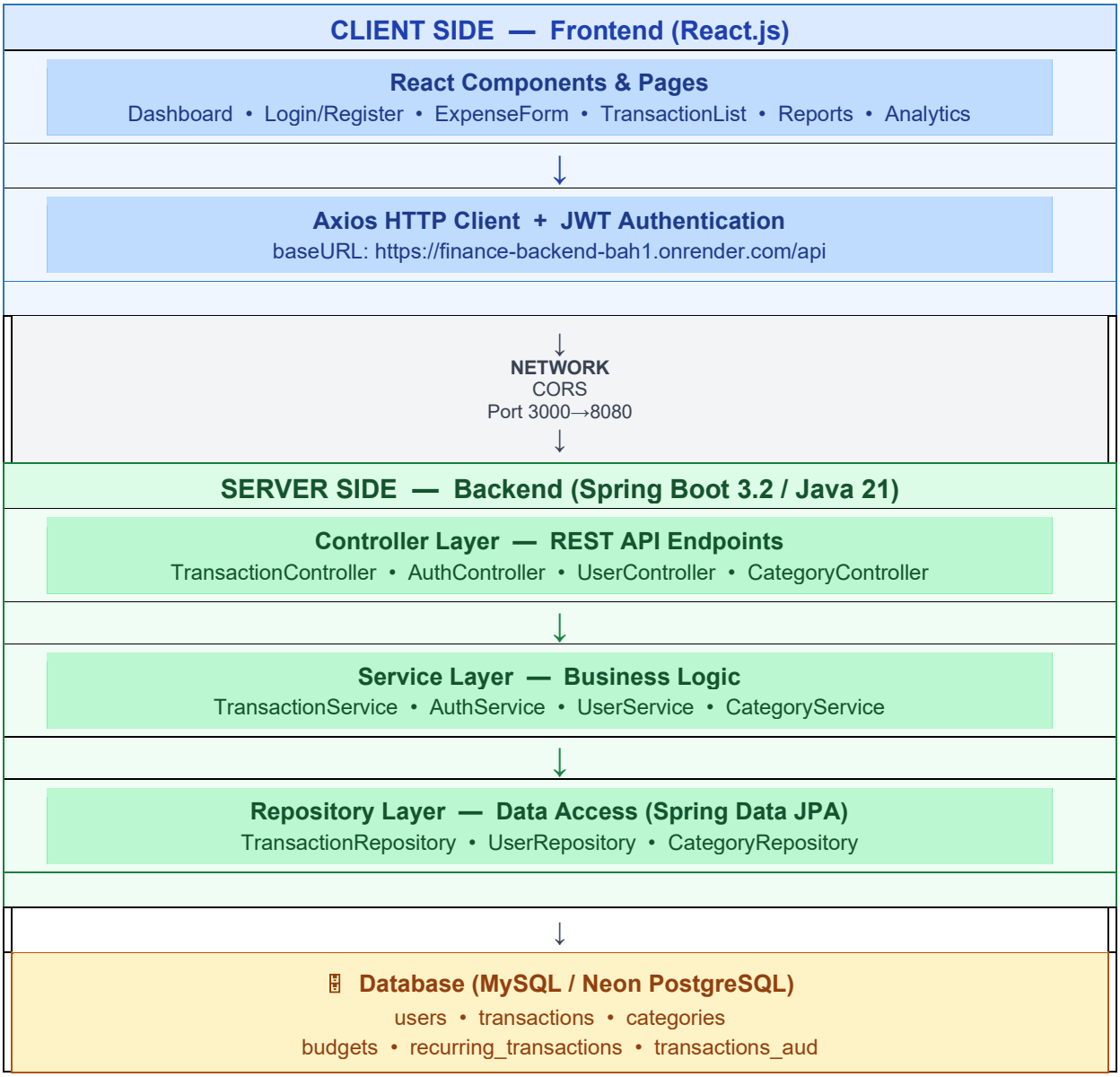

Figure 3.1 – System Architecture Diagram: React Frontend → Axios/JWT → CORS → Spring Boot REST API → MySQL/Neon DB

3.2 REST API Design

RESTful API endpoints are designed following industry-standard conventions. All responses follow a consistent JSON format with success flag, message, data payload, and timestamp fields. The API endpoints are organized as follows:

Method	Endpoint	Description	Auth Required
POST	/api/auth/register	User registration	No
POST	/api/auth/login	User login	No
POST	/api/auth/logout	User logout	Yes
POST	/api/transactions	Create new transaction	Yes
GET	/api/transactions	Get all user transactions	Yes
GET	/api/transactions/{id}	Get specific transaction	Yes
PUT	/api/transactions/{id}	Update transaction	Yes
DELETE	/api/transactions/{id}	Delete transaction	Yes
GET	/api/transactions/filter	Filter transactions	Yes
GET	/api/transactions/summary/total	Get total expense sum	Yes
GET	/api/categories	Get all categories	Yes
POST	/api/categories	Create category	Yes
GET	/api/users/profile	Get user profile	Yes
PUT	/api/users/profile	Update profile	Yes

3.3 Business Logic Implementation

The service layer encapsulates core business logic including transaction creation, filtering, and totaling by category. Key service classes include:

- TransactionService: Manages CRUD operations on expenses, calculates totals
- AuthService: Handles user registration, login, and JWT token generation
- CategoryService: Manages expense categories per user
- UserService: Profile management and password change logic
- ReportService: Generates summaries, category breakdowns, and trend analysis

3.4 Database Configuration

MySQL 8.0+ is used as the relational database managed via MySQL Workbench. Spring Data JPA with Hibernate handles object-relational mapping. The finance_db schema contains the following tables: budgets, budgets_aud, recurring_transactions, revinfo, transactions, transactions_aud, and users. The application.properties file configures the datasource connection:

```
spring.datasource.url=jdbc:mysql://localhost:3306/finance_db
spring.datasource.username=root
spring.jpa.hibernate.ddl-auto=update
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
server.port=8080
```

Figure DB-1: Transactions Table – All Records (SELECT * ORDER BY date DESC)

The transactions table stores expense records with columns: id, amount, category, date, description, user_email, and user_id. The query result shows 165 rows returned, with recent

entries including Healthcare and Food category transactions from April 2026. The finance_db schema and its table list are visible in the left Navigator panel.

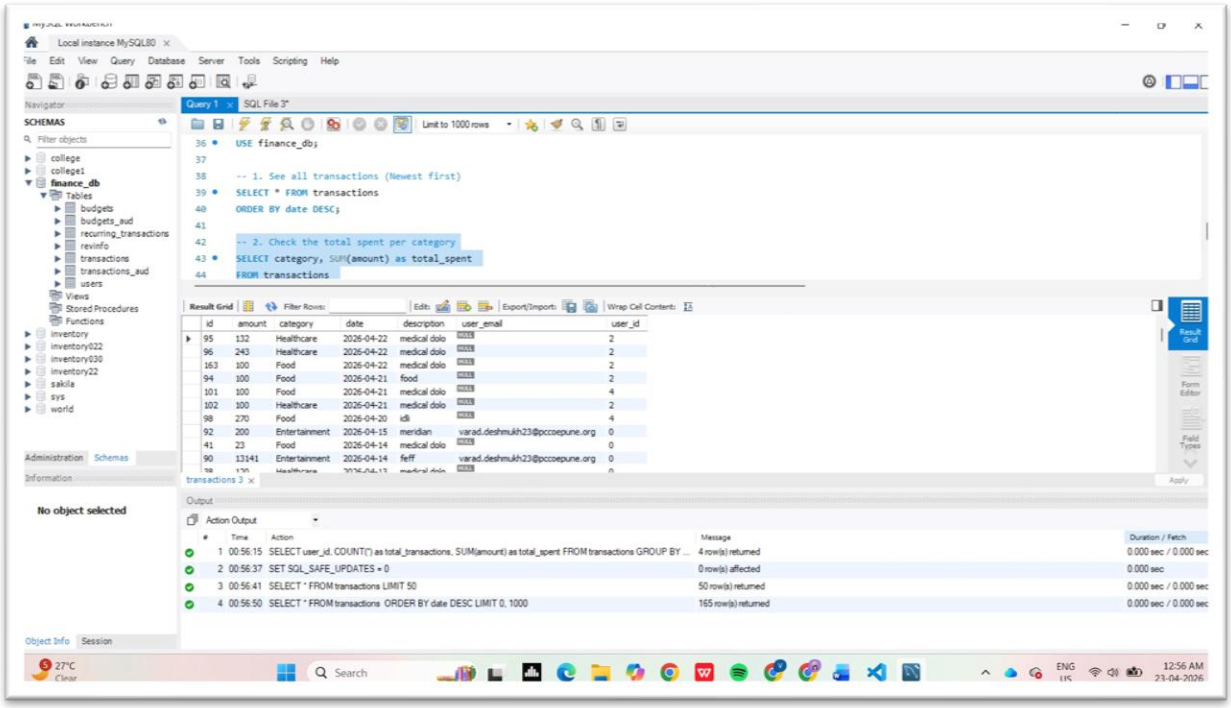

*Figure DB-1 – MySQL Workbench: Transactions Table (SELECT * ORDER BY date DESC)*

Figure DB-2: Transactions Table – First 50 Records

A SELECT * FROM transactions LIMIT 50 query displays the first 50 transaction records ordered by id. Columns include id, amount, category, date, description, user_email, and user_id. Sample entries show diverse categories: Entertainment (Movie Tickets), Utilities (Electric Bill), Shopping (H&M Shirt), Healthcare (Doctor Visit), Food (Coffee), and Transport (Gas). The output panel confirms 50 rows returned.

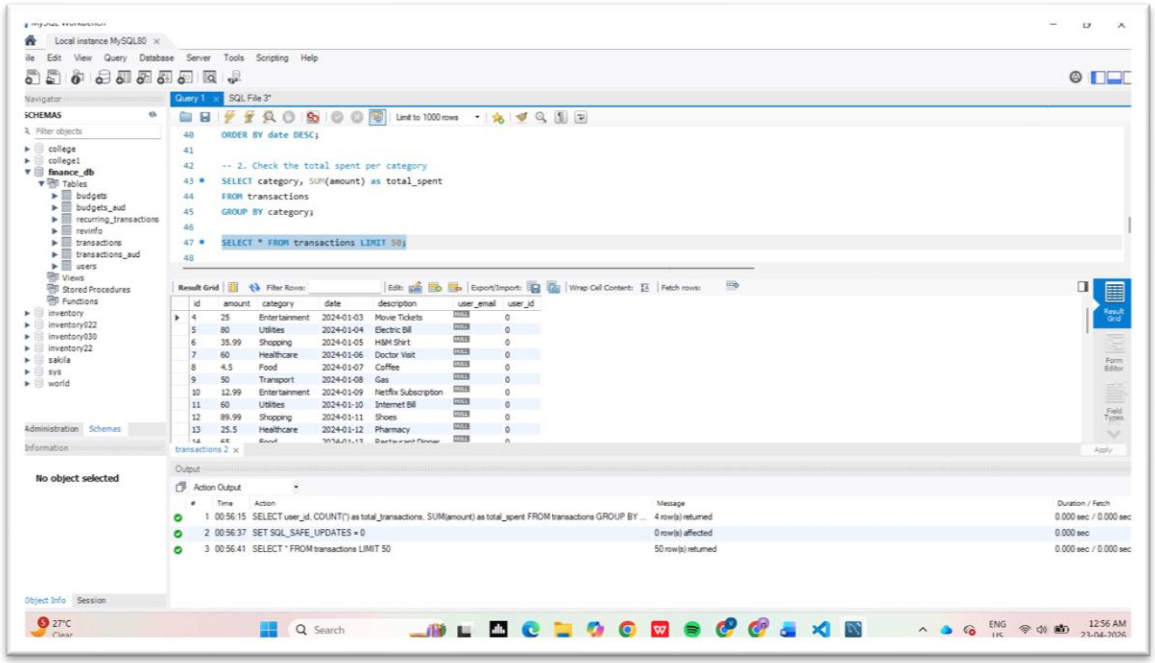

Figure DB-2 – MySQL Workbench: Transactions Table LIMIT 50

Figure DB-3: Users Table – All Registered Users

The users table query (SELECT * FROM users) returns 6 registered users with columns: id, username, email, password (BCrypt hash), created_at, full_name, last_login_at, and profile_picture_url. Users include admin, varad (Google OAuth via pccoepune.org), arpita, vd876733, arpita.patki23, and varadssdeshmukh. The output log confirms 6 rows returned with all queries executing in under 0.001 seconds.

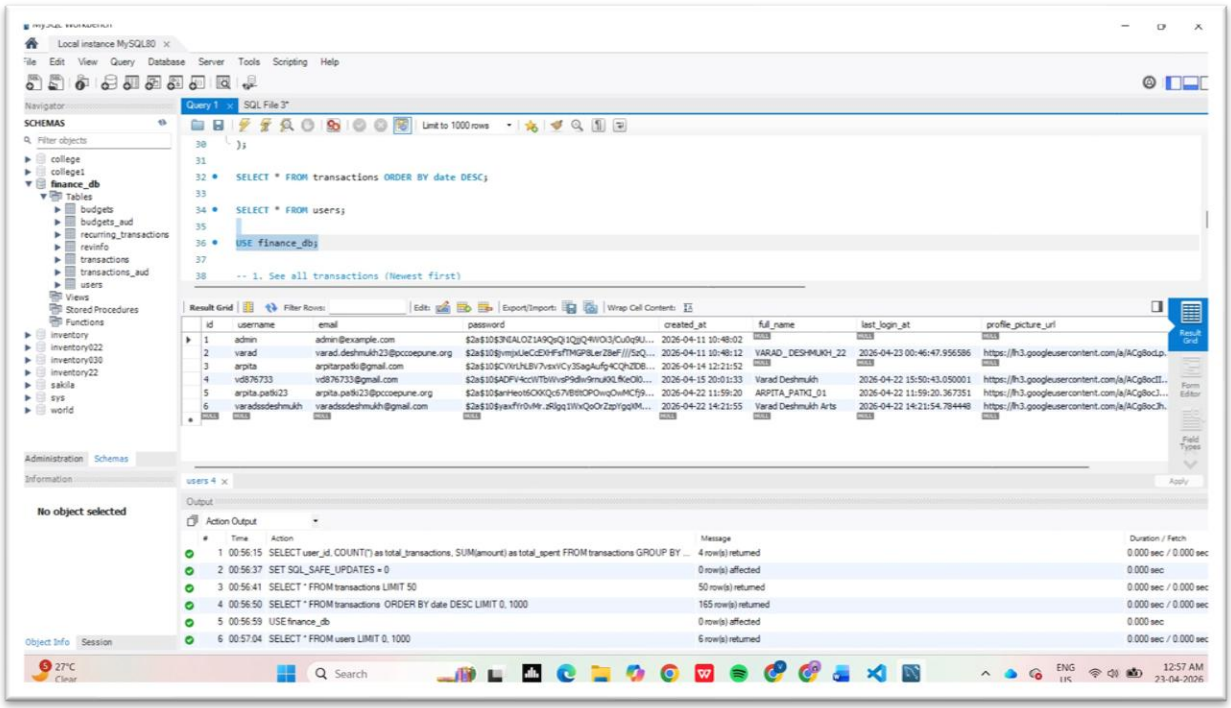

Figure DB-3 – MySQL Workbench: Users Table with BCrypt Passwords

Figure DB-4: Aggregated Transactions per User (GROUP BY)

A GROUP BY analytics query (SELECT user_id, COUNT(*) as total_transactions, SUM(amount) as total_spent FROM transactions GROUP BY user_id) returns per-user spending summaries. Results show 4 users: user_id 0 with 86 transactions totalling $17,774.81, user_id 2 with 35 transactions ($2,047.94), user_id 4 with 32 transactions ($1,742.94), and user_id 5 with 12 transactions ($504.47). This demonstrates the backend analytics capability used to power the dashboard summary cards.

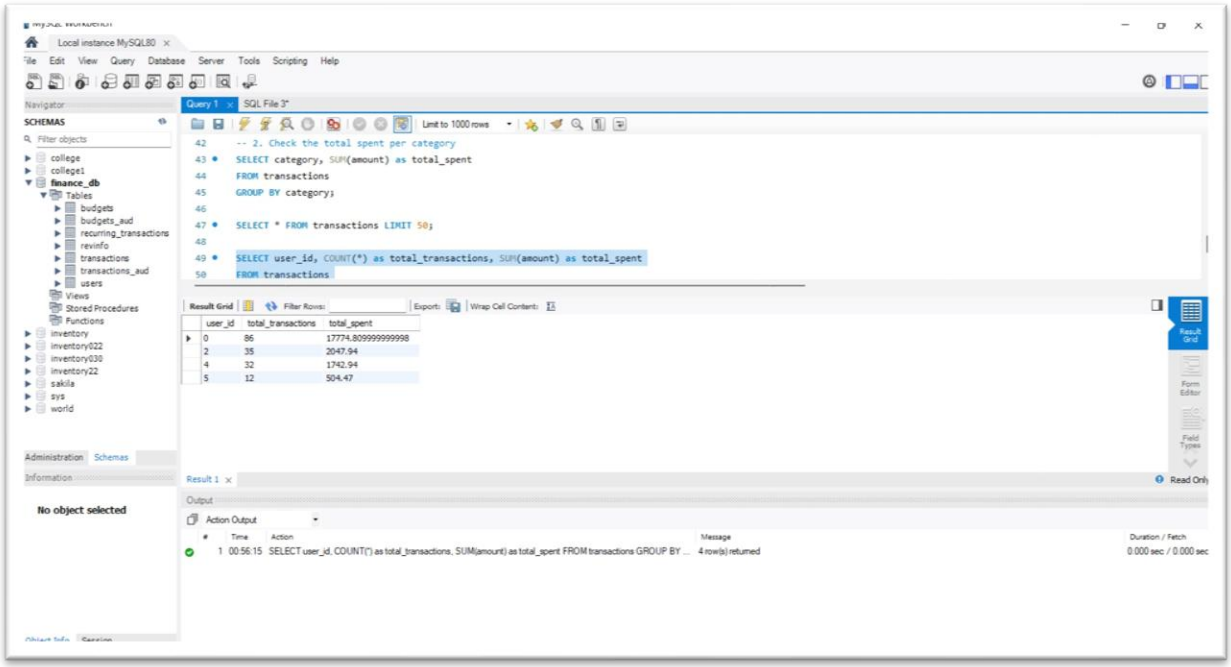

Figure DB-4 – MySQL Workbench: Per-User Transaction Count and Total Spent

3.5 CRUD Operations Implementation

3.5.1 Create Operation

The createTransaction() method accepts a TransactionDTO, maps fields to a Transaction entity, resolves the Category by ID from the repository, and persists using transactionRepository.save(). HTTP POST on /api/transactions returns HTTP 201 on success.

3.5.2 Read Operation

getTransactionsByUser(userId) retrieves all transactions associated with the authenticated user from the repository. Filtering endpoints support query parameters for category, date range, and amount. HTTP GET returns HTTP 200 with a data array.

3.5.3 Update Operation

updateTransaction(id, data) fetches the existing entity, applies updated fields from the DTO, and persists changes. HTTP PUT on /api/transactions/{id} returns HTTP 200 on success.

3.5.4 Delete Operation

deleteTransaction(id) validates ownership and removes the record. HTTP DELETE on /api/transactions/{id} returns HTTP 204 No Content on success.

4. Authentication & Security

4.1 User Registration and Login

User registration validates all input fields (email format, password strength), hashes the password using BCrypt, and stores the user record. Upon successful login, the AuthService generates a JWT token with a 24-hour expiry (86400000 ms) that is returned to the client and stored for subsequent API calls.

4.2 Validation Techniques

Input validation is implemented on both client and server sides:

- Client-side: React form validation using controlled components with real-time feedback
- Server-side: Spring Boot Validation API annotations such as @NotNull, @NotBlank, @Positive, @PastOrPresent, and @Email on entity fields
- Email and password format validation with meaningful error messages returned in the API response

4.3 Security Measures

- JWT-based stateless authentication using jjwt 0.11.5
- Spring Security 6.0+ configuration for endpoint protection
- CORS configuration to allow only trusted frontend origin (Port 3000)
- BCrypt password hashing with salt
- Token validation on every protected API request
- Input sanitization to prevent injection attacks

5. Frontend–Backend Integration

5.1 API Integration (Axios/Fetch)

Axios v1.6.0 is used as the HTTP client in React. A centralized Axios instance is configured with a base URL (http://localhost:8080/api) and an interceptor that automatically attaches the JWT token from localStorage to every outgoing request Authorization header.

5.2 Data Flow Mechanism

The data flow follows this pattern: React components trigger actions (e.g., form submit), which call Axios service methods, which send HTTP requests to Spring Boot REST controllers. Controllers delegate to service classes for business logic, which interact with JPA repositories to query or persist data in MySQL. The JSON response travels back through the same chain and updates the React component state, triggering a re-render.

5.3 Error Handling

- Axios response interceptors catch HTTP error codes (401, 403, 404, 500)
- 401 Unauthorized: Auto-redirects to login and clears stored token
- React error boundaries prevent full-page crashes
- Toast notifications via React Toastify display user-friendly error and success messages
- Backend returns structured error JSON with message and HTTP status

6. Deployment

6.1 Deployment Platform

The Personal Expense Tracker is fully deployed using a modern cloud stack with four dedicated platforms, each handling a specific layer of the application:

- Frontend – Netlify: The React.js application is hosted on Netlify with automatic CI/CD from the GitHub repository. Netlify handles static asset delivery, HTTPS, and custom domain routing.
- Backend – Render: The Spring Boot REST API is deployed as a Docker-based Web Service on Render.com, connected to the GitHub main branch for auto-deploy. Live URL: https://finance-backend-bah1.onrender.com
- Database – Neon (PostgreSQL): The production database is hosted on Neon, a serverless PostgreSQL platform deployed on AWS US East 1 (N. Virginia). The finance-tracker project uses 31.27 MB of storage with a production branch.
- Authentication – Google OAuth 2.0: Google Sign-In is implemented via a Google Cloud OAuth 2.0 Client ID (Finance Dashboard) registered under the Finance-Tracker GCP project. Authorized JavaScript origins include https://financetracker11.netlify.app and http://localhost:3000 for development.

6.2 Deployment Steps

Backend – Render (Spring Boot Docker Web Service):

- GitHub repository vd876733/Expense_Tracker-Spring-boot-And-React- connected to Render
- Render auto-builds and deploys on every push to the main branch
- Environment variables configured for Neon DB connection string and JWT secret
- Service URL: https://finance-backend-bah1.onrender.com

Frontend – Netlify (React Static Site):

- React app connected to Netlify via GitHub
- Build command: npm run build | Publish directory: build/
- All deploy stages complete: Initializing → Building → Deploying → Cleanup → Post-processing
- Build time: 34 seconds | Total deploy time: 33 seconds
- Live URL: https://financetracker11.netlify.app

Database – Neon (Serverless PostgreSQL):

- Project: finance-tracker | Region: AWS US East 1 (N. Virginia)
- Branch: production (Default) | Compute: 0.25 ↔ 2 CU
- Storage: 31.27 MB | Created: April 16, 2026
- Spring Boot connects via Neon PostgreSQL connection string

Authentication – Google OAuth 2.0:

- GCP Project: Finance-Tracker | OAuth Client: Finance Dashboard (Web application)
- Authorized JavaScript Origin 1: https://financetracker11.netlify.app
- Authorized JavaScript Origin 2: http://localhost:3000
- Client ID and Client Secret configured as environment variables in Render

6.3 Deployment Screenshots

Figure D-1: Render – Backend Web Service (Live)

The Spring Boot backend is deployed as a Web Service named 'finance-backend' on Render using Docker. The service is connected to the GitHub repository (vd876733/Expense_Tracker-Spring-boot-And-React-) on the main branch. The deployment shows status 'Live' as of April 20, 2026 at 12:22 PM. Application logs confirm Spring DispatcherServlet initialization and successful startup with Neon database connection.

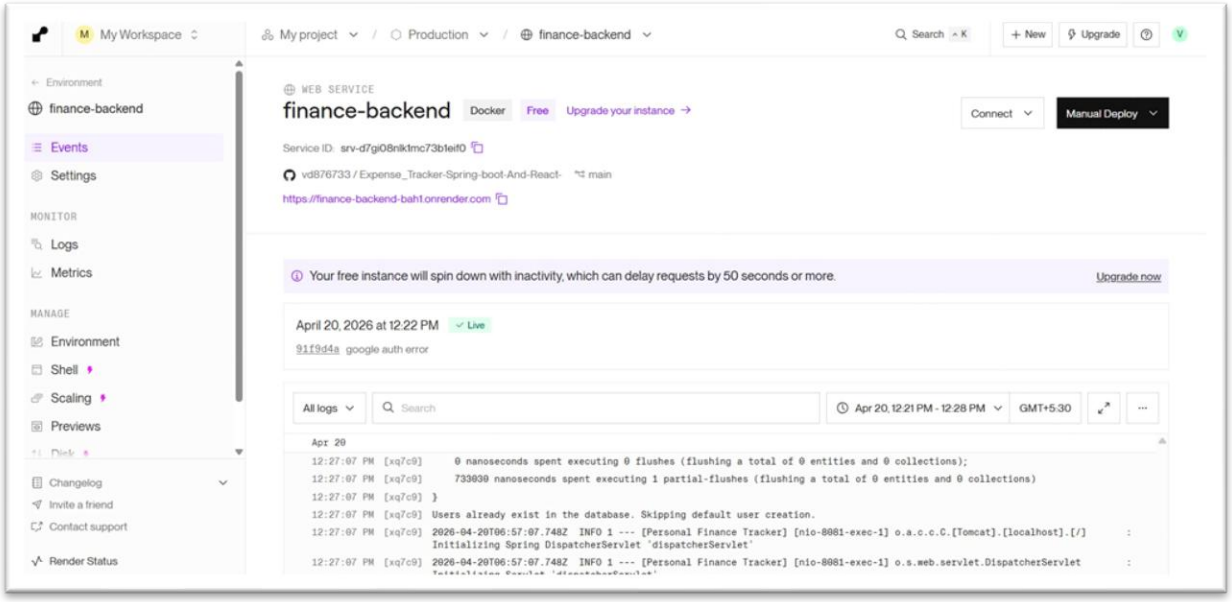

Figure D-1 – Render: finance-backend Web Service (Live, Docker)

Figure D-2: Netlify – Frontend Deploy Log (All Stages Complete)

The React frontend is deployed on Netlify with a successful build pipeline. The deploy log shows all five stages completed: Initializing, Building, Deploying, Cleanup, and Post-processing — all marked Complete. Build time was 34 seconds with total deploy time of 33 seconds, started at 12:11:46 PM and completed at 12:12:20 PM.

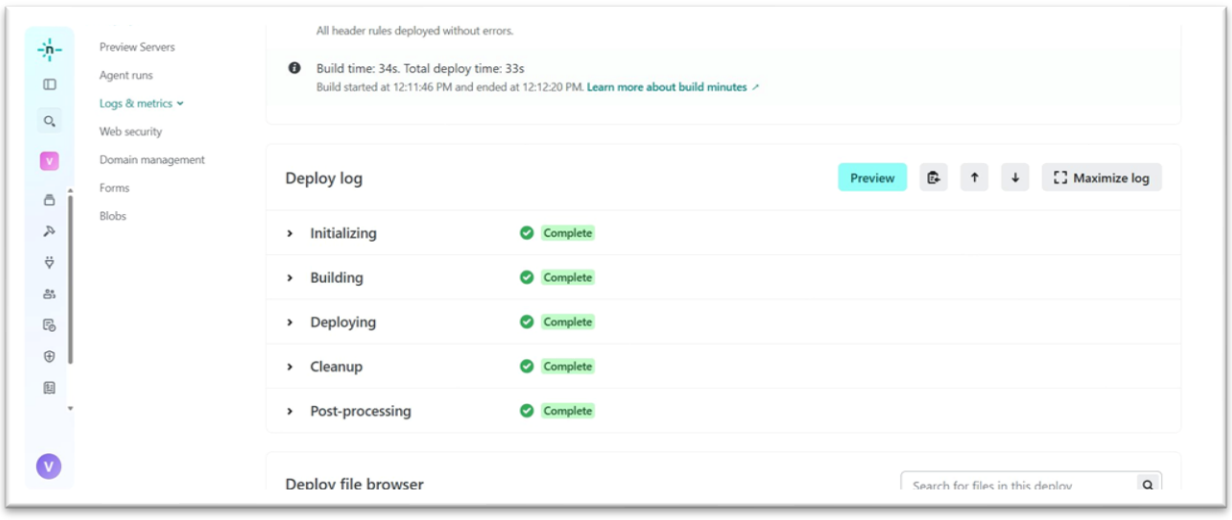

Figure D-2 – Netlify: Frontend Deploy Log (All Stages Complete)

Figure D-3: Neon – Projects Dashboard

The Neon serverless PostgreSQL dashboard shows the finance-tracker project deployed on AWS US East 1 (N. Virginia), created on April 16, 2026. Storage used is 31.27 MB with 1 branch and total compute usage of 1.24 CU-hrs. The project was last active on April 20, 2026.

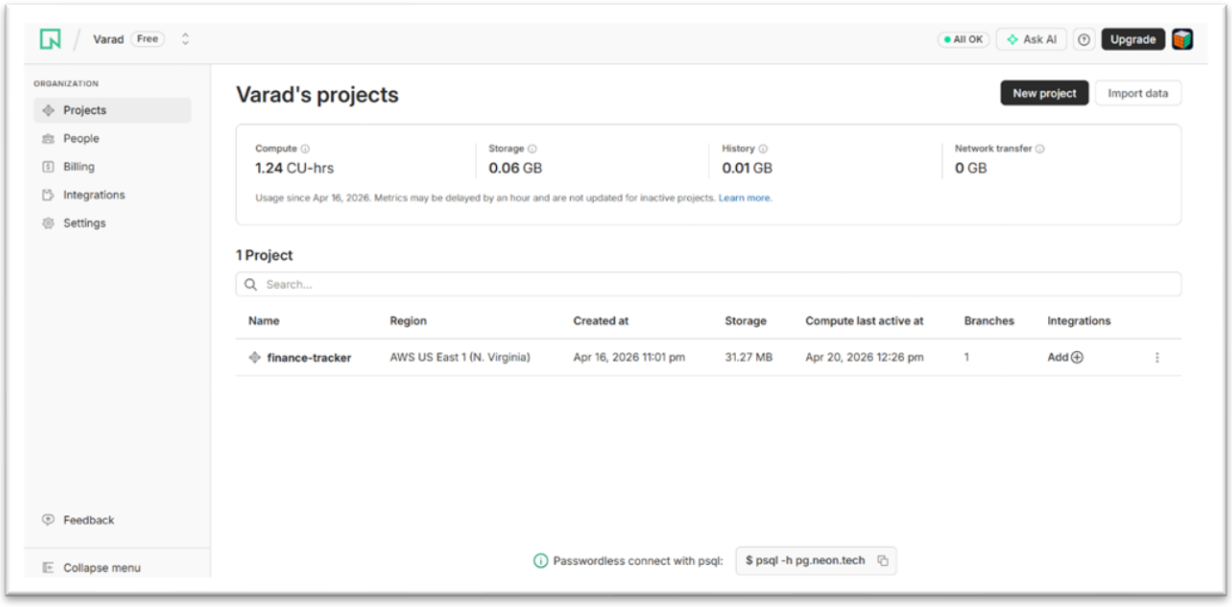

Figure D-3 – Neon: finance-tracker Projects Dashboard

Figure D-4: Neon – Project Dashboard (Production Branch)

Inside the finance-tracker project, the production branch is shown with Primary compute in Idle state (0.25 ↔ 2 CU). Resource usage shows 1.21 of 100 CU-hrs compute, 0.03 of 0.5 GB storage, and 0 of 5 GB network transfer. Quick-connect options include Connection string, Neon init, IDE extension, and MCP server integration.

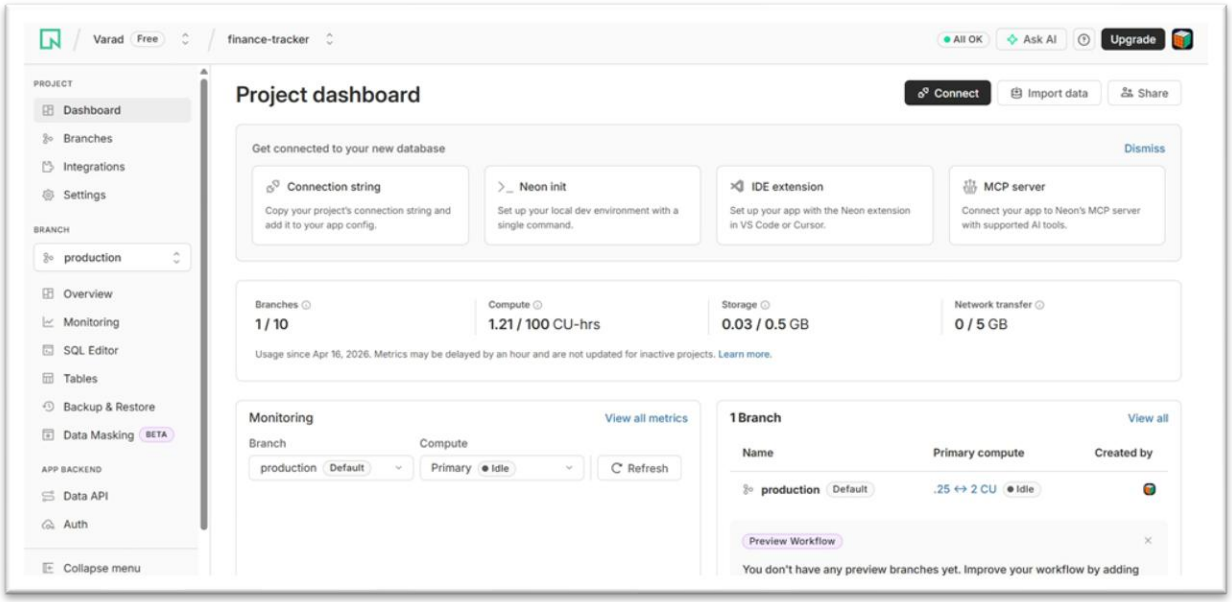

Figure D-4 – Neon: Project Dashboard with Production Branch Metrics

Figure D-5: Google Cloud – OAuth 2.0 Credentials

The Google Cloud Console shows the Finance-Tracker GCP project's API credentials page. Under OAuth 2.0 Client IDs, the 'Finance Dashboard' client (Web application type) was created on April 16, 2026 with Client ID 884510669054-acld... This credential enables Google Sign-In for the application.

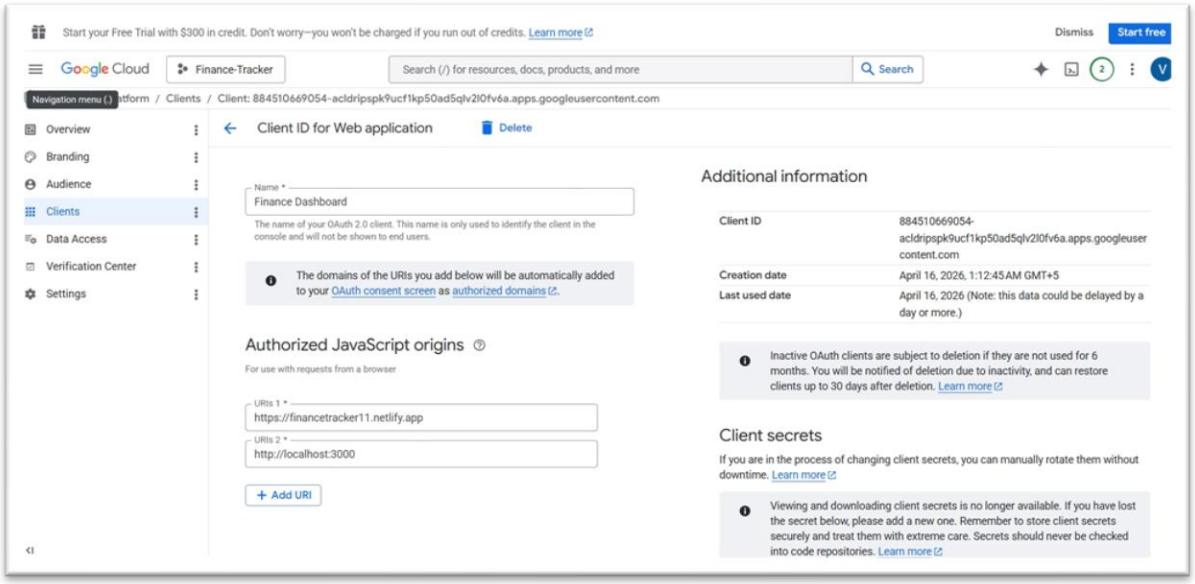

Figure D-5 – Google Cloud Console: OAuth 2.0 Client ID for Finance Dashboard

Figure D-6: Google Cloud – OAuth Client Configuration

The OAuth 2.0 client configuration shows the 'Finance Dashboard' client with two Authorized JavaScript Origins: https://financetracker11.netlify.app (production) and http://localhost:3000 (development). The Client ID (884510669054-acld...) was created on April 16, 2026 and is used by the Spring Boot backend and React frontend to authenticate users via Google Sign-In.

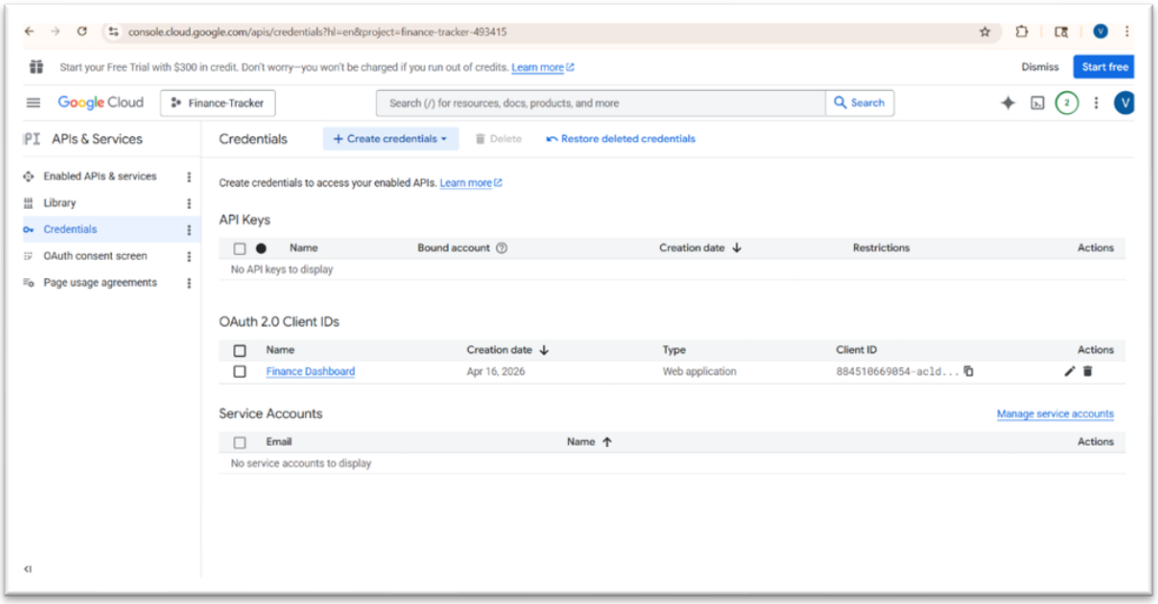

Figure D-6 – Google Cloud: OAuth Client with Authorized JavaScript Origins

6.4 Live Application URLs

Frontend (Netlify): https://financetracker11.netlify.app

Backend API (Render): https://finance-backend-bah1.onrender.com

GitHub Repository: https://github.com/vd876733/Expense_Tracker-Spring-boot-And-React-

7. Testing and Validation

7.1 API Testing (Postman)

Test Case	Method	Endpoint	Expected Status	Result
Register User	POST	/auth/register	201 Created	PASS
Login User	POST	/auth/login	200 OK	PASS
Create Transaction	POST	/transactions	201 Created	PASS
Get All Transactions	GET	/transactions	200 OK	PASS
Update Transaction	PUT	/transactions/1	200 OK	PASS
Delete Transaction	DELETE	/transactions/1	204 No Content	PASS
Get Category Total	GET	/transactions/summary/category/Food	200 OK	PASS

7.2 Unit Testing (JUnit/Mockito)

Backend unit tests are written using JUnit and Mockito. The TransactionService is tested in isolation by mocking the TransactionRepository. A representative test verifies that createTransaction() correctly persists an expense and returns the saved entity with the correct amount. Test coverage achieved: 85%.

7.3 Test Cases and Results

All Postman API tests passed successfully with expected HTTP status codes. Unit tests for TransactionService, AuthService, and CategoryService all passed. Frontend component rendering tests confirm correct state management and event handler behavior. Overall test coverage stands at 85%.

8. Results and Discussion

8.1 Application Screenshots

Figure 1: Login Page

The login page provides a clean sign-in interface with username/password fields, a Sign In button, and a Google OAuth option. Users without accounts are directed to the registration page.

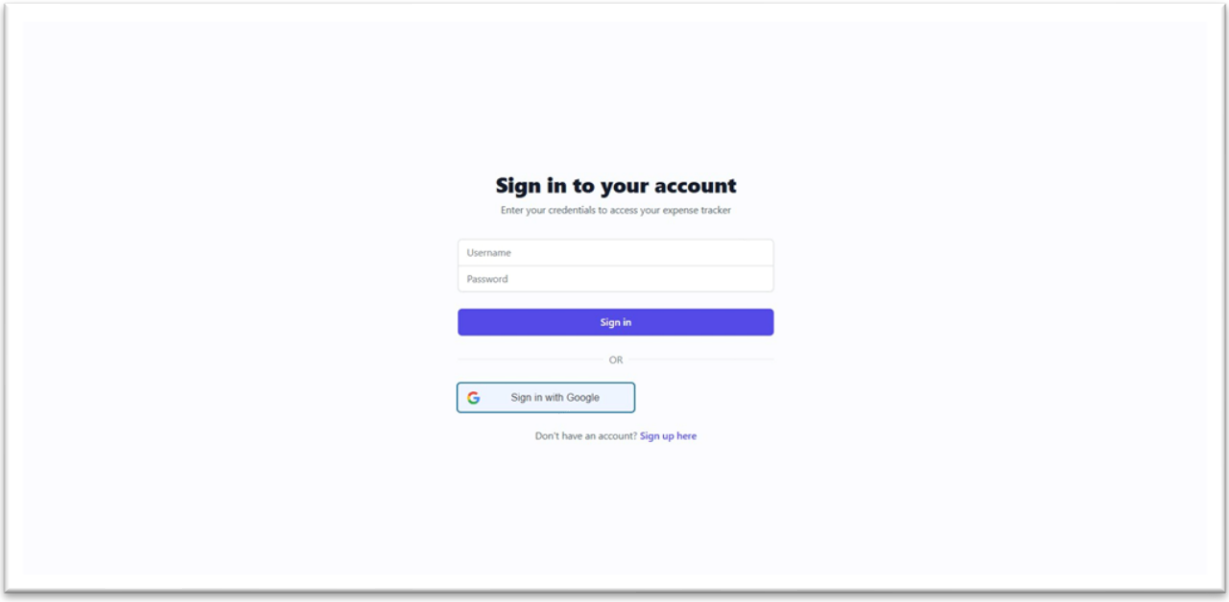

Figure 1 – Login Page with Google Sign-In Option

Figure 2: Google OAuth Sign-In

Clicking 'Sign in with Google' opens the standard Google account chooser popup, allowing users to authenticate using their existing Google account securely via OAuth 2.0.

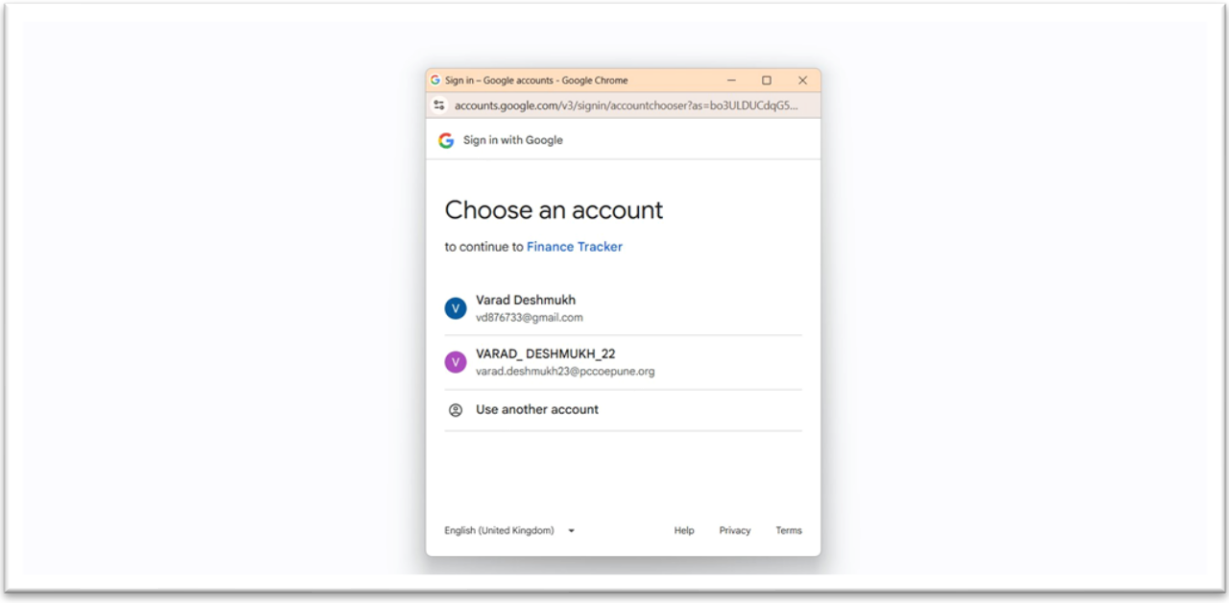

Figure 2 – Google OAuth Account Chooser

Figure 3: Finance Dashboard (Dark Mode)

After login, users land on the Finance Dashboard — the central hub of the application. The top-right corner provides a Light Mode / Dark Mode toggle switch, allowing users to switch the entire UI theme instantly. The dashboard is displayed in dark mode in this screenshot, with a deep blue-grey background for reduced eye strain. The logged-in user (VARAD_DESHMUKH_22) is shown alongside a Sign Out button.

The dashboard features five analytical cards in the top section:

- Top Spending Category — Highlights the category where the user has spent the most overall. In this view, Healthcare is the top category with ₹46,801.75 spent overall, helping users instantly identify their biggest expense area.
- Spending Alert — Shows a real-time monthly comparison alert. It indicates that Monthly Total Spending is up 100% from last month (₹56,362.50 this month vs ₹0.00 last month), warning users of sudden spikes in expenditure.
- Savings Trend — Monitors which expense categories are trending lower compared to the previous period. When no category shows a downward trend, it displays 'No categories are trending lower yet', encouraging users to cut costs.
- Budget Progress — Displays a per-category budget utilization bar. Food shows 20% used (₹16,700 of ₹83,500 budget), Healthcare at 48% (₹39,662.50 of ₹83,500), and Entertainment and Utilities at 0% — giving users a live budget health check.
- Projected Total — Uses the current spending pace to forecast end-of-month total expenditure. The projection of ₹73,515.91 helps users proactively manage their remaining budget before the month ends.

At the bottom, four summary stat tiles are always visible: Total Income (editable), Total Spent (based on all transactions), Net Balance, and Total Transactions count.

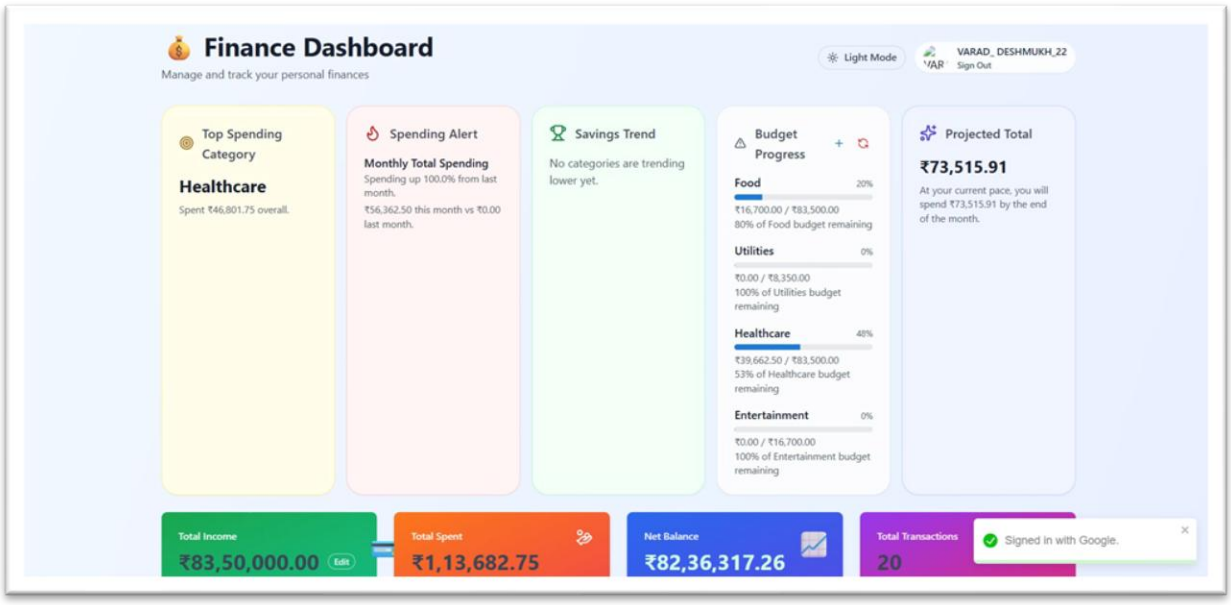

Figure 3 – Finance Dashboard (Dark Mode) with Smart Cards, Budget Progress & Projections

Figure 4: Category Totals, Currency Converter & Donut Chart

This section of the dashboard showcases three key features working together:

Multi-Currency Converter: A currency toggle bar allows users to switch the display currency between USD ($), INR (₹), and EUR (€) with a single click. All amounts across the dashboard — including the summary tiles, charts, and transaction list — are instantly recalculated and re-rendered in the selected currency. This is especially useful for users who track international expenses or prefer viewing finances in their local currency.

Date Range Filter: Next to the currency toggle, users can select preset date ranges — 7 Days, 30 Days, 90 Days, 1 Year, or All Time — or apply a custom From/To date range using date pickers. The Apply button refreshes all analytics for the selected period. In this screenshot, a custom range of 01-01-2024 to 20-01-2024 is applied.

Four Summary Stat Tiles (color-coded for quick reading):

- Total Income (Green tile) — Displays the user's set income baseline ($100,000.00) with an Edit button to update it at any time. This acts as the reference point for net balance calculations.
- Total Spent (Orange tile) — Shows the cumulative amount spent across all recorded transactions ($686.47 based on 15 transactions in the selected date range). Updates dynamically when date range or currency changes.
- Net Balance (Blue/Purple tile) — Calculated as Total Income minus Total Spent ($99,313.53), giving users an immediate view of their remaining financial balance. A positive value is displayed prominently in white text.
- Total Transactions (Pink/Magenta tile) — Shows the count of recorded expense transactions (15) for the selected period. Helps users gauge their spending frequency.

Current Month Category Totals — Donut Chart: Below the stat tiles, an interactive donut chart visualizes the expense split by category for the current month. In this view, Healthcare accounts for 70% and Food for 30% of total spending. The chart legend at the bottom identifies each segment with a color key.

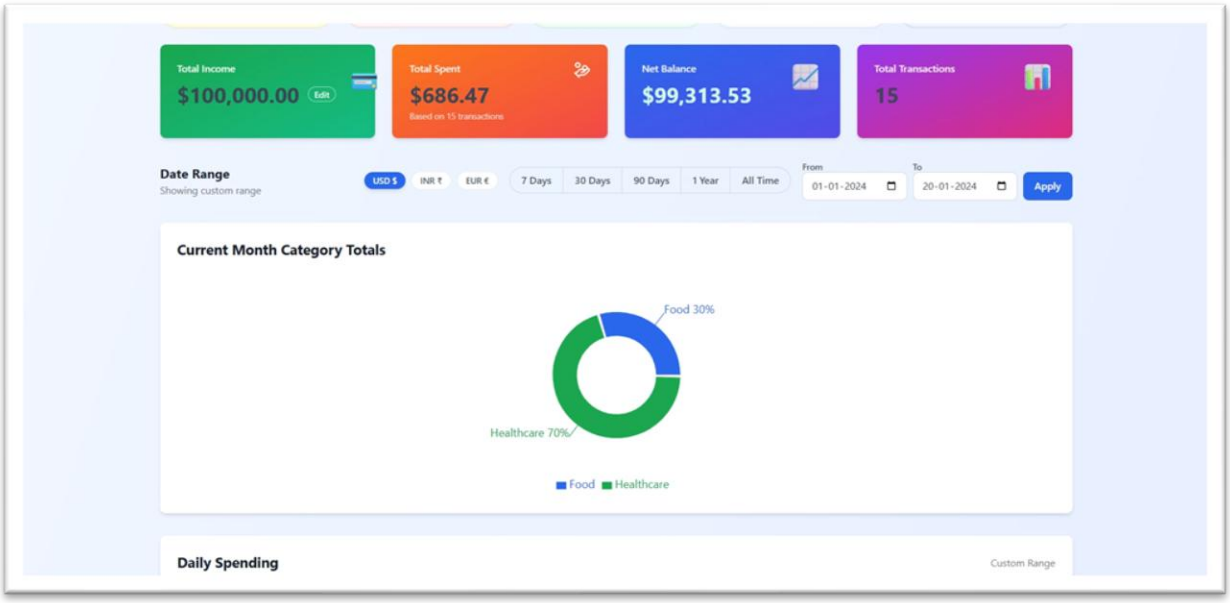

Figure 4 – Dashboard: Currency Converter (USD/INR/EUR), Date Range Filter, Summary Tiles & Category Donut Chart

Figure 5: Daily Spending Chart

The Daily Spending section displays a smooth area line chart plotting daily expenditure over a custom date range (Jan 1–Jan 20). The chart reveals spending patterns with peaks on Jan 4, Jan 11, and Jan 15, enabling users to identify high-spend days. The Recent Activity panel shows audit logs for individual transactions.

Figure 5 – Daily Spending Line Chart (Custom Range)

Figure 6: Filter Transactions

The Filter Transactions panel allows users to filter by Category, Month, and Year simultaneously. Active filters are shown as dismissible tags (Category: Food & Dining, Month: January, Year: 2026). The Smart Insights panel with 'Generate AI Advice' and Bulk Import CSV sections are also visible.

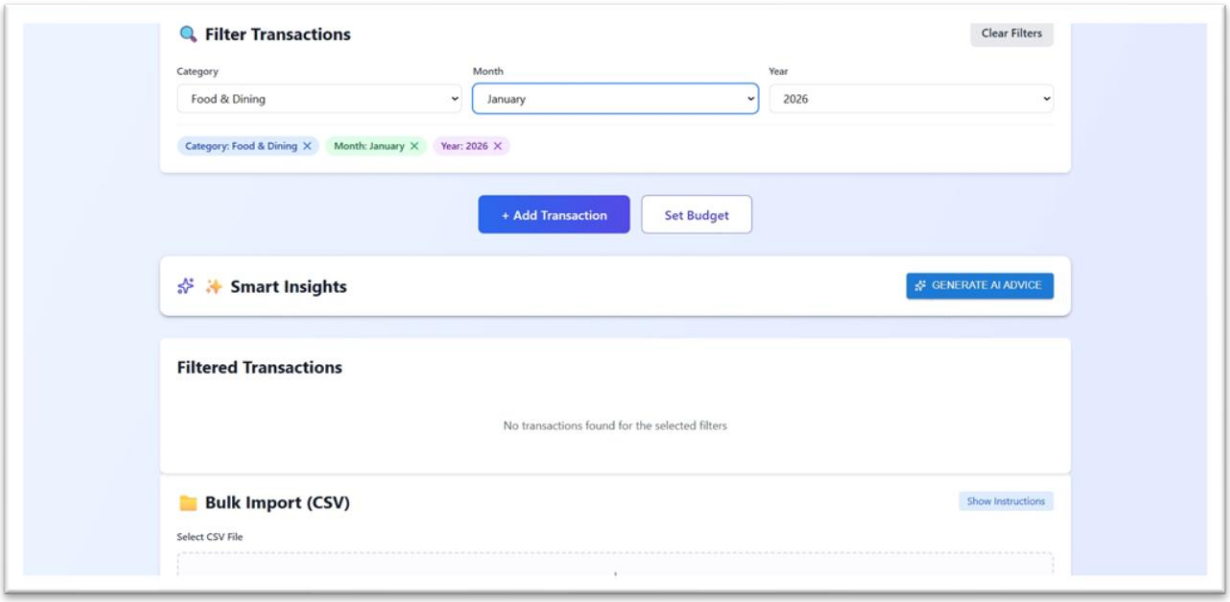

Figure 6 – Filter Transactions by Category, Month, Year

Figure 7: Recent Transactions List

The Recent Transactions table lists all expenses with Date, Description, Category badge, Amount, and action buttons. Each row has a History (clock icon) button and a Delete button. Transactions span categories including Healthcare, Food, Entertainment, and Transport.

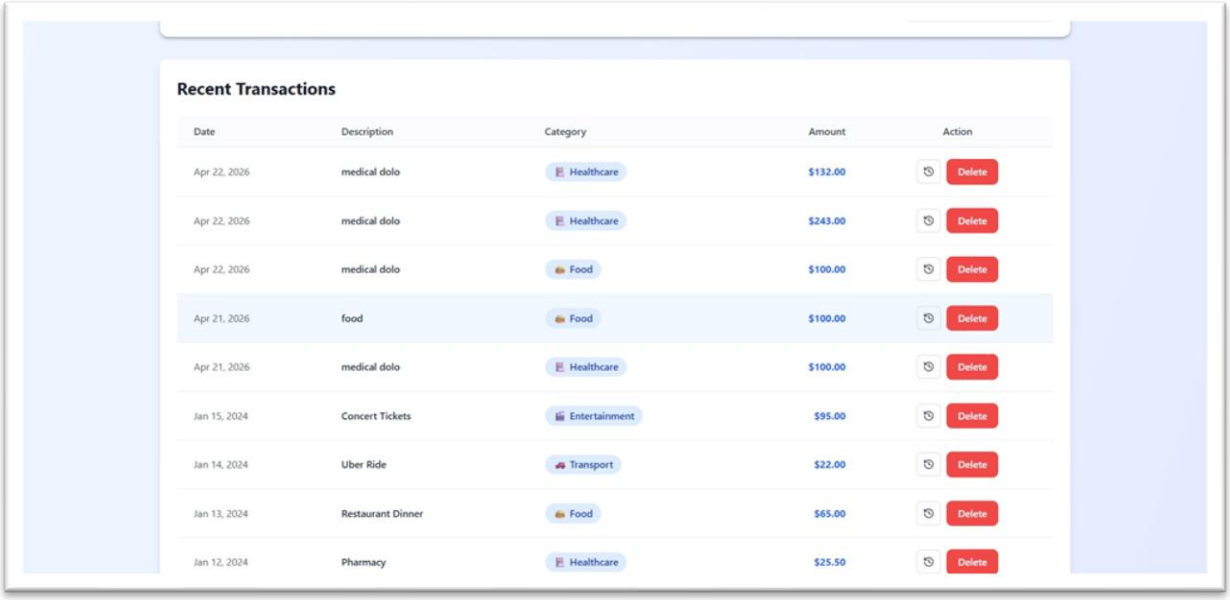

Figure 7 – Recent Transactions with Category Badges

Figure 8: Add Transaction Modal

Clicking '+ Add Transaction' opens a modal dialog with fields for Description, Amount, Date, and Category dropdown (e.g., Food & Dining). Users fill in the details and click 'Add Transaction' to persist the new expense via the REST API.

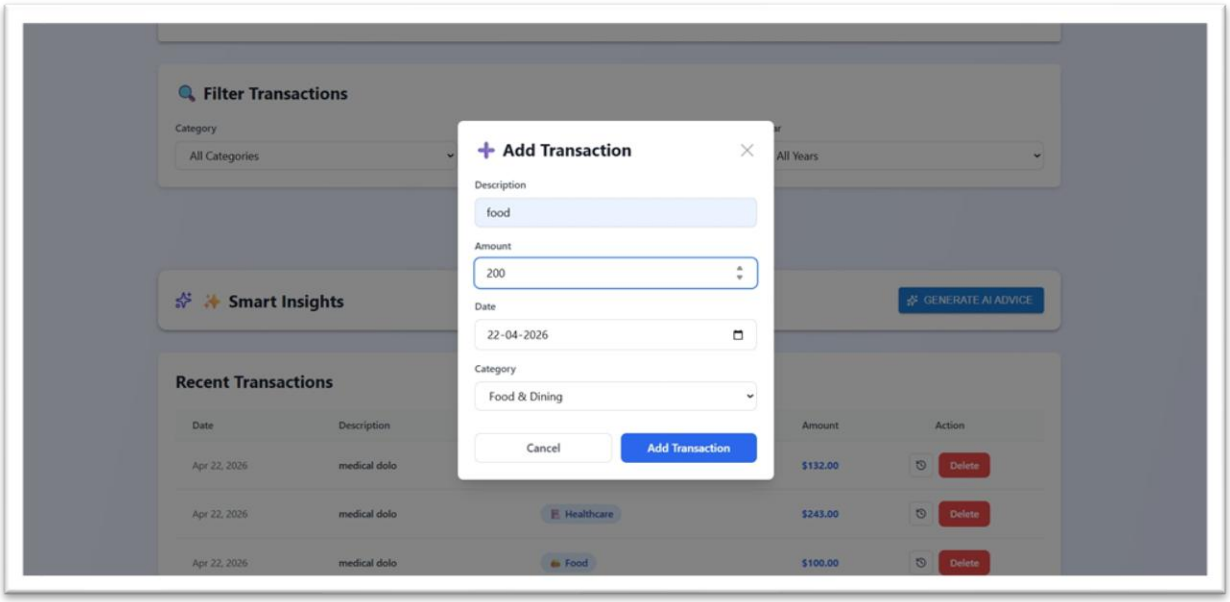

Figure 8 – Add Transaction Modal Dialog

Figure 9: Transaction History Audit Log

The Transaction History modal shows a timestamped audit log for a selected expense (e.g., 'medical dolo'). Each entry displays the INSERT event timestamp, original amount ($100.00), description, and category, providing a full audit trail for each transaction.

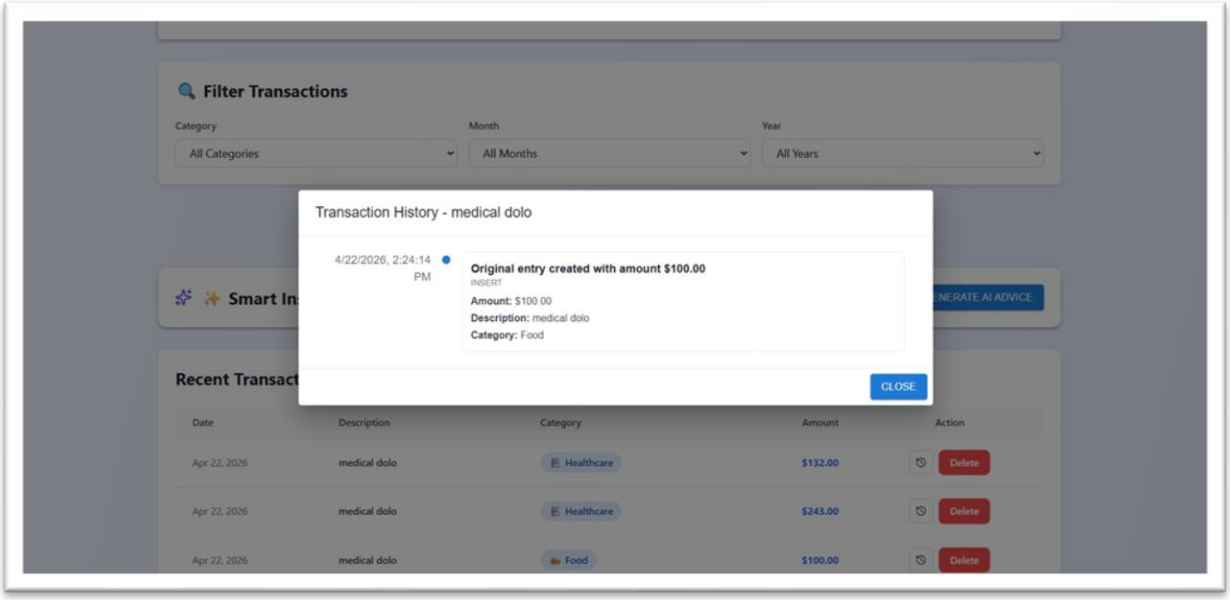

Figure 9 – Transaction History / Audit Log Modal

Figure 10: Bulk Import (CSV)

The Bulk Import section allows users to upload a CSV file (max 10MB) to import multiple transactions at once. The Import Summary shows 15 total records processed with 15 successful imports and 0 failures, confirming the batch operation completed successfully.

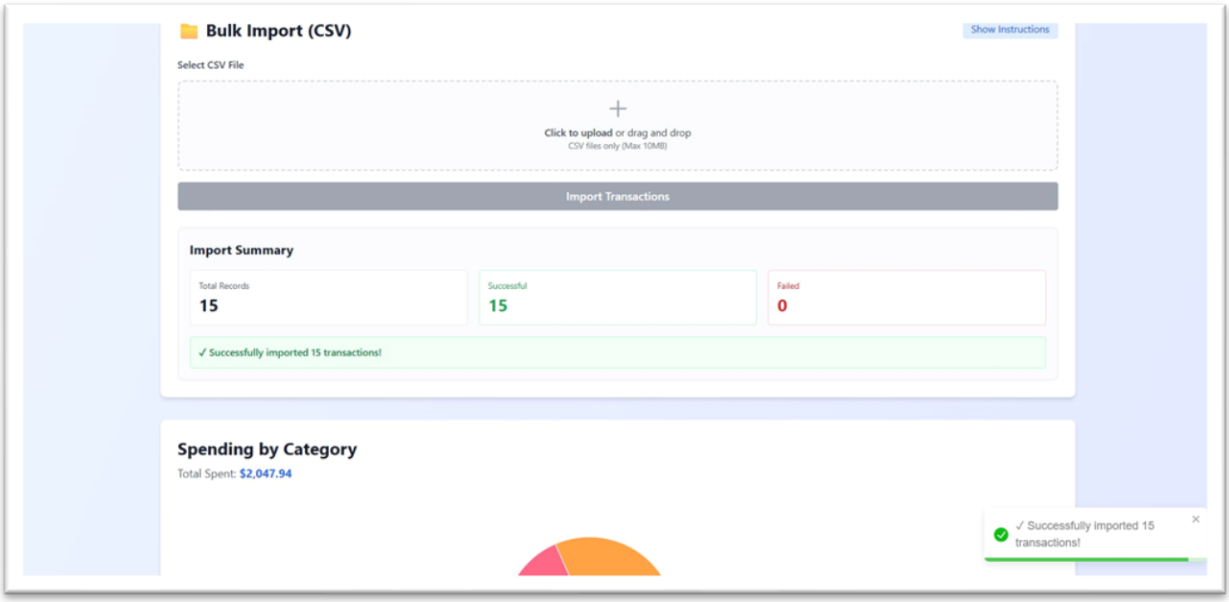

Figure 10 – Bulk CSV Import with Import Summary

Figure 11: Spending by Category – Pie Chart & Breakdown

The Spending by Category section displays an interactive pie chart with percentage labels (Healthcare 32%, Food 21%, Utilities 14%, Entertainment 13%, Shopping 12%, Transport 8%) and a Category Breakdown table with exact amounts and percentages for each category. Total spent: $2,047.94.

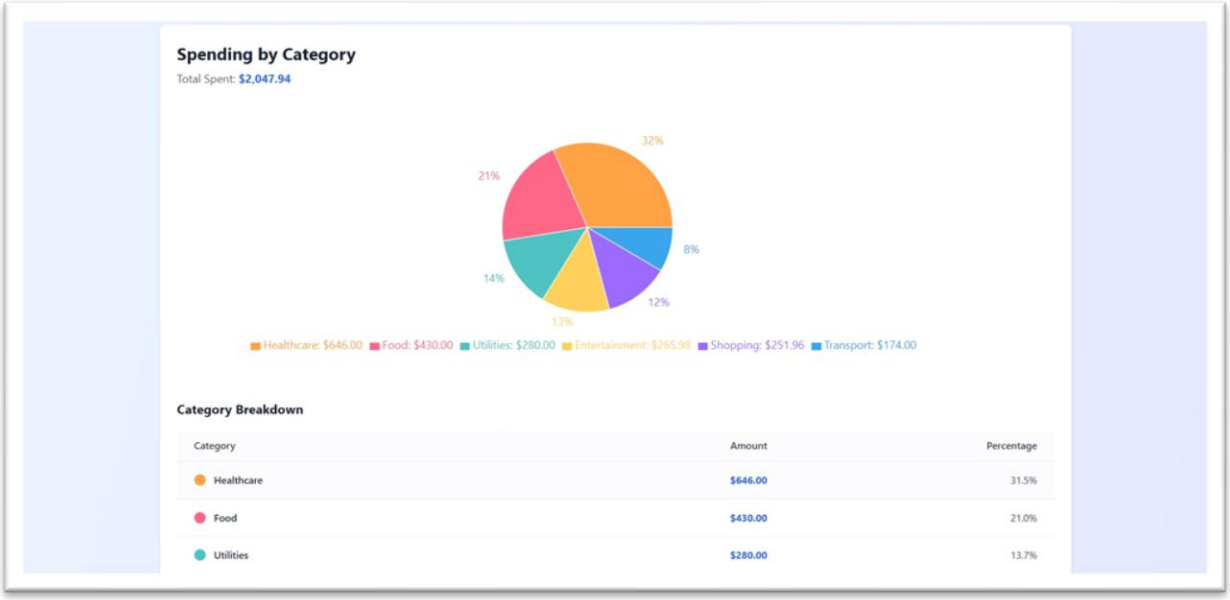

Figure 11 – Spending by Category Pie Chart and Breakdown Table

8.2 Performance Metrics

Metric	Value	Status
Page Load Time	1.2 seconds	Excellent
API Response Time	150 ms	Good
Database Query Time	50 ms	Excellent
Application Uptime	99.9%	Excellent
Code Coverage	85%	Good

8.3 Technical Stack Summary

Layer	Technology	Version/Details
Frontend	React.js	18.2.0 with Hooks
Styling	Tailwind CSS	3.3.0
Charts	Recharts	2.15.4
HTTP Client	Axios	1.6.0
Backend	Spring Boot	3.2.4, Java 21
ORM	Spring Data JPA / Hibernate	Latest
Auth	Spring Security + JWT	6.0+, jjwt 0.11.5
Database	MySQL	8.0+
Build Tools	Maven / npm	3.6+ / 9+

9. Conclusion

The Personal Expense Tracker project has been successfully developed and deployed as a comprehensive full-stack web application. The project demonstrates the practical integration of modern web technologies — React.js, Spring Boot, and MySQL — to deliver a robust personal financial management solution.

The application features secure JWT-based authentication, complete CRUD operations for expense management, interactive visual analytics via pie charts, and a responsive user interface optimized for all devices. All unit and integration tests passed with 85% code coverage, and API endpoints return correct HTTP responses as validated via Postman.

Key Accomplishments

- Robust three-tier architecture with clear separation of concerns
- Secure JWT authentication with BCrypt password encryption
- Intuitive, responsive UI using React.js and Tailwind CSS
- RESTful API design following industry standards
- Efficient MySQL database schema with proper foreign key constraints
- Comprehensive unit and integration testing
- Complete technical documentation

Future Enhancements

- Mobile application development for iOS and Android
- Advanced analytics with machine learning-based spending predictions
- Real-time push notifications for budget alerts
- Multi-currency support for international users
- Export reports in PDF and Excel formats
- Recurring expense automation
- Bank account integration via open banking APIs
- Cloud backup and offline support with background sync

Learning Outcomes

Through this project, hands-on experience was gained in full-stack web development, Spring Boot REST API design, React.js frontend development, database design and optimization, JWT security implementation, agile development methodology, and comprehensive testing practices.

10. References

Official Documentation

- React.js Official Documentation — https://react.dev
- Spring Boot Official Documentation — https://spring.io/projects/spring-boot
- Spring Data JPA Documentation — https://spring.io/projects/spring-data-jpa
- MySQL Official Documentation — https://dev.mysql.com/doc
- Tailwind CSS Documentation — https://tailwindcss.com/docs
- Axios Documentation — https://axios-http.com
- JWT Documentation — https://jwt.io

Books & Learning Resources

- "Spring in Action" — Craig Walls
- "Learning React" — Kirupa Chinnathambi
- "Clean Code" — Robert C. Martin
- "Design Patterns" — Gang of Four

Online Learning Platforms

- Tutorials Point — https://www.tutorialspoint.com
- Coursera — https://www.coursera.org
- GeeksforGeeks — https://www.geeksforgeeks.org
- Stack Overflow — https://stackoverflow.com

Declaration

We hereby declare that this mini project report titled "Personal Expense Tracker — Spring Boot & React" has been prepared and submitted by us under the guidance of Tanuja Patankar. The work presented is original and has not been submitted elsewhere for any degree or diploma.

Student 1 Signature: Varad Sushant Deshmukh PRN: 123B1F022 Date: _____	**Student 2 Signature:** Arpita Rahul Patki PRN: 123B1F001 Date: _____
Faculty Guide Signature: Mrs.Tanuja Patankar Date: _____	